

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl



Estructura de Datos en Pandas: Series vs. DataFrame

Pandas Series y Dataframe

Estructura unidimensional etiquetada. Funciona como una columna de una planilla o un vector numérico con un índice de búsqueda. El Dataframe es una estructura bidimensional (tablas) compuesta por múltiples Series alineadas por un mismo índice.

Ejemplo de Código

```
import pandas as pd

# Series
ventas_mensuales = pd.Series([1200, 1350, 1100], index=['Ene', 'Feb', 'Mar'])

# DataFrame
df_gestion = pd.DataFrame({
    'Mes': ['Ene', 'Feb', 'Mar'],
    'Ventas': [1200, 1350, 1100],
    'Costos': [800, 850, 790]
})
```

Indexación Avanzada: .loc vs .iloc

`.loc[filas, columnas]`

Acceso basado en **etiquetas/nombres** de filas y columnas. Incluye los límites dados.

`.iloc[filas, columnas]`

Acceso basado en **posiciones enteras** (índices numéricos de 0 a $N - 1$). Excluye el límite final.

Ejemplo de Uso

```
# Seleccionar filas específicas por etiqueta de índice y columnas
df_gestion.loc[df_gestion['Mes'] == 'Feb', ['Ventas', 'Costos']]

# Seleccionar las primeras 2 filas y las primeras 2 columnas
df_gestion.iloc[0:2, 0:2]
```

Introducción a NumPy para Control de Gestión

- **Cómputo Numérico de Alto Rendimiento:** Base matemática subyacente de Pandas.
- **Procesamiento Vectorizado:** Operaciones simultáneas sobre arrays sin iterar con bucles (for).
- **Manejo de Nulos:** `np.nan` define valores faltantes compatibles con IEEE Floating Point standard.

Operaciones Vectorizadas

```
import numpy as np

ingresos = np.array([1000, 1500, 2000])
costos = np.array([600, 900, 1100])
margen_pct = (ingresos - costos) / ingresos
```

Condicionales Vectorizados con np.where()

Equivalente a SI() de Excel

Aplica condicionales lógicos evaluando toda la columna de manera instantánea.

```
# Sintaxis: np.where(condicion, valor_si_verdadero, valor_si_falso)
df['Margen_Pct'] = (df['Ventas'] - df['Costos']) / df['Ventas']

df['Status_Margen'] = np.where(
    df['Margen_Pct'] >= 0.25,
    'Objetivo Cumplido',
    'Revisar Estructura de Costos'
)
```

Reglas Multicriterio con np.select()

Anidado Eficiente de Condicionales

Supera la limitación de múltiples SI() anidados en Excel ofreciendo claridad sintáctica y rapidez de ejecución.

```
condiciones = [  
    df['Ventas'] >= 100000,  
    (df['Ventas'] >= 50000) & (df['Ventas'] < 100000),  
    df['Ventas'] < 50000  
]  
  
valores = ['Tier 1 (Alto)', 'Tier 2 (Medio)', 'Tier 3 (Bajo)']  
  
df['Clasificacion_Cliente'] = np.select(condiciones, valores, default='Sin Clasificar')
```

Tratamiento de Valores Faltantes (NaN)

Atención

Los valores nulos corrompen los cálculos numéricos y distorsionan los reportes financieros.

- **Detección:** `df.isna().sum()`
- **Eliminación:** `df.dropna(subset=['Ventas'])`
- **Imputación:** `df['Gastos'].fillna(df['Gastos'].median(), inplace=True)`

```
# Identificar nulos por columna  
print(df.isna().sum())
```

```
# Reemplazar valores nulos en presupuestos por 0 o promedio  
df['Presupuesto'] = df['Presupuesto'].fillna(0)
```

Modificación de Tipos de Datos

Casting de Datos

Suele suceder que montos numéricos o fechas ingresen como strings (object).

```
# Limpiar caracteres especiales de texto a numero
df['Monto'] = df['Monto'].str.replace('$', '').str.replace(',', '').astype(float)

# Conversion estandar a tipo DateTime
df['Fecha_Transaccion'] = pd.to_datetime(df['Fecha'], format='%Y-%m-%d')
```

Agrupación de Datos con groupby()

Estrategia Split-Apply-Combine

Divide la tabla según categorías, aplica una función de agregación y combina el resultado.

```
# Total de gastos por Gerencia
gastos_por_gerencia = df.groupby('Gerencia')['Gastos'].sum()

# Promedio de ventas por Region
ventas_promedios = df.groupby('Region')['Ventas'].mean()
```

Agrupaciones Multivariable y Multi-métrica

Agregación Compleja con .agg()

Permite aplicar distintas funciones estadísticas a diferentes columnas simultáneamente.

```
resumen_ejecutivo = df.groupby(['Gerencia', 'Centro_Costo']).agg(  
    Ventas_Totales=('Ventas', 'sum'),  
    Margen_Promedio=('Margen', 'mean'),  
    Transacciones=('ID', 'count')  
)
```

Tablas Dinámicas con pivot_table()

Modelado Multidimensional

Genera vistas cruzadas idénticas a las Tablas Dinámicas (Pivot Tables) de Excel.

```
pivot_ventas = df.pivot_table(  
    index='Gerencia',  
    columns='Mes',  
    values='Gastos',  
    aggfunc='sum',  
    fill_value=0,  
    margins=True,           # Total General  
    margins_name='Total',  
)
```

Fusión de DataFrames con merge()

Relaciones de Bases de Datos

Equivalente funcional a BUSCARV() o a las cláusulas JOIN en SQL.

```
# Cruce entre Transacciones y Maestro de Vendedores
df_consolidado = pd.merge(
    left=df_ventas,
    right=df_vendedores,
    on='ID_Vendedor',
    how='left'
)
```

Tipos de Joins en `pd.merge()`

Tipo de Join	Comportamiento
<code>left</code>	Conserva todas las filas de la tabla izquierda.
<code>right</code>	Conserva todas las filas de la tabla derecha.
<code>inner</code>	Retiene solo registros con clave en ambas tablas.
<code>outer</code>	Mantiene todos los registros de ambas tablas.

```
# Ejemplo Inner Join
df_coincidencias = pd.merge(df_presupuesto, df_real, on='Cod_CC', how='inner')
```

Concatenación de DataFrames con `pd.concat()`

Unión Estructural

Apilado vertical (añadir registros) u horizontal (añadir atributos).

```
# Concatenacion Vertical (Apilar archivos mensuales id nticos)
lista_meses = [df_enero, df_febrero, df_marzo]
df_q1 = pd.concat(lista_meses, axis=0, ignore_index=True)

# Concatenacion Horizontal
df_completo = pd.concat([df_datos, df_metricas_extra], axis=1)
```

Manipulación de Fechas con el Accesor `.dt`

Componentes Temporales

Facilita la extracción de atributos clave para análisis de estacionalidad y cierres periódicos.

```
df['Fecha'] = pd.to_datetime(df['Fecha'])

df['Anio'] = df['Fecha'].dt.year
df['Mes'] = df['Fecha'].dt.month
df['Trimestre'] = df['Fecha'].dt.quarter
df['Dia_Semana'] = df['Fecha'].dt.day_name()
```

Análisis de Variaciones Temporales (YoY, MoM)

Cálculo de Desviaciones

Evaluación de crecimiento porcentual respecto al periodo previo.

```
# Ordenar cronologicamente antes de calcular
df = df.sort_values('Fecha')

# Variacion mensual porcentaje MoM
df['Var_MoM_%'] = df['Ventas'].pct_change() * 100

# Diferencia absoluta
df['Dif_Absoluta'] = df['Ventas'].diff()
```

Manipulación de Cadenas de Texto con .str

Estandarización de Atributos

Limpia inconsistencias en nombres de Centros de Costos, Clientes o Productos.

```
# Limpieza en cadena
df['Centro_Costo'] = df['Centro_Costo'].str.strip().str.upper()

# Búsqueda de patrones
df['Es_Sucursal'] = df['Centro_Costo'].str.contains('SUC_')
```

Filtrado Avanzado mediante isin()

Selección por Lista de Criterios

Optimiza la sintaxis evitando múltiples condicionales OR (|).

```
cc_objetivos = ['CC01_ADMIN', 'CC05_VENTAS', 'CC10_IT']  
  
# Filtrado directo  
df_filtrado = df[df['Centro_Costo'].isin(cc_objetivos)]
```

Normalización y Renombrado de Columnas

```
# Renombrar específicamente
df.rename(columns={
    'Vta_Net': 'Ventas_Netas',
    'Fch': 'Fecha_Factura'
}, inplace=True)

# Estandarizacion masiva
df.columns = df.columns.str.lower().str.replace(' ', '_')
```

Manejo de Duplicados en Registros Transaccionales

```
# Identificar duplicados
duplicados = df.duplicated(subset=['ID_Transaccion'], keep=False)

# Eliminar duplicados manteniendo el ultimo registro cargado
df_limpio = df.drop_duplicates(subset=['ID_Transaccion'], keep='last')
```

Cálculo de Acumulados YTD (cumsum())

```
# Ordenar por periodo
df = df.sort_values(['Anio', 'Mes'])

# Acumulado Year-To-Date por linea de negocio
df['Ventas_YTD'] = df.groupby(['Anio', 'Linea_Negocio'])['Ventas'].cumsum()
```

Funciones Personalizadas con `apply()`

Flexibilidad Algorítmica

Permite ejecutar funciones personalizadas o expresiones lambda sobre filas o columnas.

```
def calcular_impuesto(row):  
    if row['País'] == 'Chile' and row['Utilidad'] > 0:  
        return row['Utilidad'] * 0.27  
    return 0  
  
df['Impuesto_Estabilidad'] = df.apply(calcular_impuesto, axis=1)
```

Resúmenes Estadísticos Rápidos

```
# Resumen general de distribucion estadistica
print(df['Resultado'].describe())

# Resumen agrupado
df.groupby('Linea_Negocio')['Resultado'].describe()
```

Restablecimiento de Índices (`reset_index()`)

Aplanar DataFrames

Convierte el índice resultante de agrupaciones o pivotes de nuevo en columnas estándar.

```
df_resumen = df.groupby('Gerencia')['Gastos'].sum().reset_index()
```

- **Categorías:** Convertir variables de texto repetitivas a category.
- **Downcasting:** Reducir precisión numérica (float64 a float32).
- **Uso de inplace:** Usar con prudencia para modificar estructuras sin duplicar objetos en RAM.

```
df['Gerencia'] = df['Gerencia'].astype('category')  
df.info(memory_usage='deep')
```

Caso Práctico Integrador: Cierre Clase 2

Desafío de Control de Gestión

Unir maestro de sucursales con transacciones mensuales, evaluar el cumplimiento del presupuesto acumulado (YTD) e identificar desviaciones por Zona Geográfica.

```
# 1. Unir Maestro y Transacciones
df_full = pd.merge(df_transacciones, df_sucursales, on='ID_Sucursal', how='left')

# 2. Agrupar y Calcular Acumulados YTD
df_full['Ventas_YTD'] = df_full.groupby(['Zona', 'Anio'])['Ventas'].cumsum()
df_full['Presupuesto_YTD'] = df_full.groupby(['Zona', 'Anio'])['Presupuesto'].cumsum()

# 3. Calculo de Desviacion
df_full['Desviacion_YTD_%'] = ((df_full['Ventas_YTD'] - df_full['Presupuesto_YTD']) /
                                df_full['Presupuesto_YTD']) * 100
```

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl

